DEVELOPER GUIDE

Car Wash Management System

Odoo 19 Community Edition — Architecture, Data Model, Security & Integration Reference

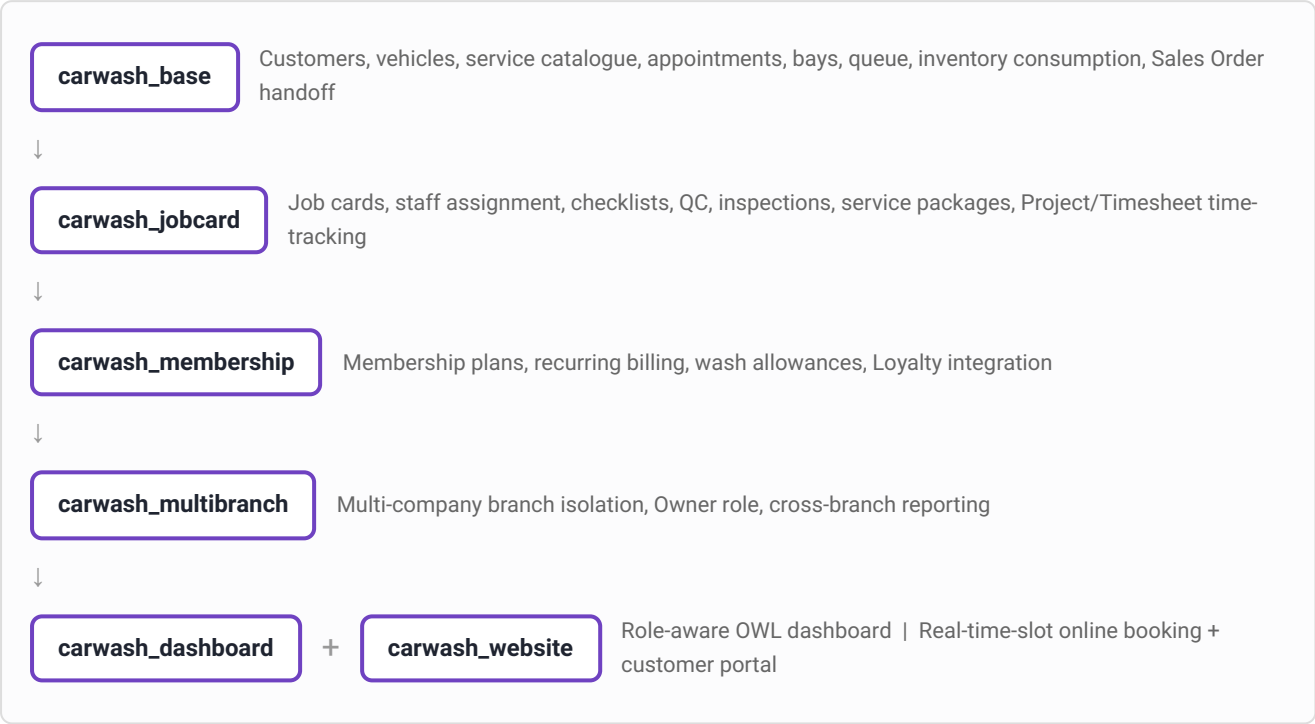
Modules	**6 (carwash_base, carwash_jobcard, carwash_membership, carwash_multibranch, carwash_dashboard, carwash_website)**
Platform	**Odoo 19.0 Community Edition (no Enterprise dependencies)**
Scope	**POS is explicitly out of scope for this system**
Audience	**Developers extending, maintaining, or integrating with this system**

Table of Contents

1. Module Architecture
2. Data Model
3. Security & Access Control
4. State Machines
5. Booking & Slot-Availability Design
6. Integration Points
7. Installation & Operations

1. Module Architecture

The system is a chain of six installable Odoo apps, each depending on the previous. Every module is Community-edition only — no Enterprise modules appear anywhere in the dependency graph.



Module	Odoo Dependencies	Adds
`carwash_base`	`base, mail, contacts, product, sale_management, stock`	Core operational models
`carwash_jobcard`	`carwash_base, hr, hr_timesheet`	Job cards, QC, Project/Timesheet integration
`carwash_membership`	`carwash_base, loyalty`	Recurring billing, Loyalty points
`carwash_multibranch`	`carwash_membership`	Owner role, branch isolation rules
`carwash_dashboard`	`carwash_multibranch`	OWL client-action dashboard (no report engine)
`carwash_website`	`carwash_multibranch, website, portal, resource`	Public booking controllers, portal "My Bookings"

Why this order matters: each module only ever depends on modules earlier in the chain, so a deployment can stop at any point — e.g. a single-branch shop can skip `carwash_multibranch` / `carwash_dashboard` / `carwash_website` entirely and run just base + jobcard + membership.

2. Data Model

carwash_base

Model	Purpose	Key Relations
`carwash.appointment`	A booked or walk-in visit	customer_id → res.partner, vehicle_id → carwash.vehicle, bay_id → carwash.bay, line_ids → carwash.appointment.line, sale_order_id → sale.order
`carwash.appointment.line`	One service on an appointment	product_id → product.product (is_carwash_service=True)
`carwash.vehicle`	A customer's registered vehicle	customer_id → res.partner; unique(license_plate, company_id)
`carwash.bay`	A physical wash bay/tunnel	company_id → res.company (branch-scoped)
`carwash.service.material.line`	Consumables recipe for a service product	lives on product.template (Car Wash page)

carwash_jobcard

Model	Purpose	Key Relations
`carwash.jobcard`	The shop-floor work order for an appointment	appointment_id → carwash.appointment (cascade), assigned_employee_ids → hr.employee (m2m), checklist_template_id → carwash.checklist.template, task_id → project.task
`carwash.jobcard.checklist.line`	One task item on a job card	jobcard_id → carwash.jobcard (cascade)
`carwash.checklist.template / .item`	Reusable task lists (e.g. "Basic Wash Checklist")	template_id → carwash.checklist.template (cascade)
`carwash.inspection / .photo`	Pre/post-wash vehicle condition record	appointment_id → carwash.appointment (cascade)
`carwash.service.package / .line`	Bundled services applied to an appointment in one click	package_id → carwash.service.package (cascade)

carwash_membership

Model	Purpose	Key Relations

`carwash.membership`	A customer's active subscription	partner_id → res.partner, plan_id → carwash.membership.plan, company_id → res.company
`carwash.membership.plan`	A billable recurring plan (price, period, wash allowance, covered services)	product_id → product.product (billing product)

carwash_dashboard / carwash_website

Neither module adds new persistent business models. `carwash.dashboard` is a transient, non-stored model that exists only to expose one server method (`get_dashboard_data()`) to an OWL client action — there is nothing to query it directly for. `carwash_website` adds no models at all; it is controllers + security + templates over the existing `carwash.appointment` / `carwash.vehicle` models.

Cross-Module Extension Pattern

Later modules extend `carwash.appointment` via `_inherit` rather than new models — e.g. `carwash_jobcard` adds `jobcard_ids` / `inspection_count` and overrides `action_ready()` to gate on job-card completion; `carwash_multibranch` adds no fields at all, only `ir.rule` records. This keeps the base model's own file the single source of truth for the core fields and state machine.

3. Security & Access Control

The Five Roles

All five groups share one `res.groups.privilege` ("Car Wash", defined in `carwash_base`), which is the Odoo 19 replacement for the deprecated `category_id` grouping — it's what makes them appear together as one radio-button set on a user's form.

Group (XML ID)	Defined In	Implies	Typical User
Front Desk `group_carwash_user`	carwash_base	base.group_user	Reception / booking desk
Washer/Detailer `group_carwash_washer`	carwash_jobcard	base.group_user	Wash bay staff
QC Inspector `group_carwash_qc`	carwash_jobcard	base.group_user	Quality control staff
Manager `group_carwash_manager`	carwash_base	group_carwash_user	Branch/shop manager
Owner `group_carwash_owner`	carwash_multibranch	group_carwash_manager, base.group_multi_company	Multi-branch operator

Model Access (ir.model.access.csv)

Model	Front Desk	Washer	QC	Manager
`carwash.appointment`	RWC	R	R	RWCU
`carwash.vehicle`	RWC	—	—	RWCU
`carwash.bay`	RW	—	—	RWCU
`carwash.jobcard`	—	RW*	RW*	RWCU
`carwash.dashboard`	R	R	R	R

R=read, W=write, C=create, U=unlink. * Washer/QC job-card access is further narrowed by an `ir.rule` ("washer sees only their own assigned job cards") — see below. Owner inherits Manager's full grant.

> **Real bug found and fixed this round:** in Odoo 19, a `groups="..."` attribute inside a view's `<button>` / `<page>` arch is resolved at *runtime* against fully-qualified `module.xml_id` strings — unlike `<menuitem groups="...">`, which resolves bare names relative to the current module via the standard `ref()` mechanism. A bare `groups="group_carwash_user"` inside a form/kanban arch silently resolves to *no group at all*, which hides the element from **every user, including the superuser** (view-level group filtering is not bypassed by `env.su`). Always fully-qualify arch-embedded `groups=`, e.g. `groups="carwash_base.group_carwash_user"`, even when the group lives in the same module as the view.

Record Rules (ir.rule)

> **Combination gotcha:** Odoo ANDs together rules that have no `groups` field ("global" rules) with everything else, but ORs together rules that share a group. This system deliberately writes branch-isolation and ownership rules as *global* rules with a self-exempting domain (e.g. `... if not user.has_group('...group_carwash_owner') else []`) rather than as group-specific rules — a group-specific "Owner sees everything" rule would *OR* with a washer's "only assigned job cards" rule instead of narrowing it, defeating the restriction entirely for any user in both groups.

Rule	Model	Kind	Effect
Branch Isolation ×6	vehicle, bay, appointment, jobcard, inspection, membership	Global	`company_id in company_ids` unless Owner
Washer Own Job Cards	carwash.jobcard	Group (washer)	only job cards where they're an assigned employee
Portal Own Bookings	carwash.appointment	Group (portal)	`customer_id = user.partner_id`
Portal Own Vehicles	carwash.vehicle	Group (portal)	`customer_id = user.partner_id`

Portal users get zero direct `create` / `write` ACL on either model — every booking is created via `sudo()` inside the controller after server-side validation, so there is exactly one code path that can create a booking, not two that can drift apart.

4. State Machines

carwash.appointment.state

```
draft → confirmed → arrived → in_service → ready → done
                                                 \→ cancelled
                                    confirmed \→ no_show
```

- `action_confirm()` — requires at least one service line.
- `action_arrive()` — stamps `arrived_at` (drives the "waiting time" KPI).
- `action_start_service()` — requires a free bay; sets the bay to `occupied`; stamps `service_started_at`. `carwash_jobcard` also auto-creates a `carwash.jobcard` here.
- `action_ready()` — frees the bay. `carwash_jobcard` overrides this to *block* until every open job card on the appointment has reached `done` / `cancel` (QC-gated).
- `action_done()` — creates stock consumption moves and a confirmed Sales Order (both idempotent — skipped if already present).

Bay time-window guard: `_check_bay_time_conflict` (a `@api.constrains`) rejects any `confirmed` / `arrived` / `in_service` appointment whose `[start, start+duration)` window overlaps another appointment on the same bay in one of those three states. This supersedes an earlier, narrower version of the constraint that only checked `in_service` — the physical `bay.state != 'free'` guard in `action_start_service()` is unrelated and still applies on top of it (one guards the paper reservation, the other the physical bay).

carwash.jobcard.stages

```
draft → in_progress → qc → done
                      ^          \→ in_progress  (QC Fail / rework loop)
             draft/in_progress → cancel
```

- `action_start()` — requires `assigned_employee_ids`; auto-creates a `project.task` on first start (idempotent).
- `action_submit_qc()` — blocked while any checklist line is unchecked.
- `action_qc_pass()` — stamps `qc_inspector_id` / `end_time`; auto-creates one `account.analytic.line` timesheet entry (`end_time − start_time`, attributed to the first assigned employee).
- `action_qc_fail()` — sends the card back to `in_progress` and resets every checklist line to unchecked.

carwash.membership.state

```
active → expired
active → cancelled
```

`_bill_cycle()` (called from a cron and from `action_renew_now()`) creates+confirms a Sales Order for the plan's billing product, advances `next_billing_date` by one period, and resets `washes_used`. Each covered wash increments `washes_used` via the appointment-done hook.

5. Booking & Slot-Availability Design

Business Hours

Reused wholesale from `resource.calendar` (via `res.company.resource_calendar_id`, brought in by the `resource` / `hr` dependency chain) — zero new fields or models. Each branch's hours are managed through the standard Employees → Working Schedules screen.

_get_available_slots(company, date, service)

An `@api.model` method on `carwash.appointment` that walks each `resource.calendar.attendance` window for the requested weekday in half-hour increments, and for each candidate slot counts how many of the branch's active bays are *not* already covered by an overlapping `confirmed` / `arrived` / `in_service` appointment. Returns `[{start, bay_count, bay_ids}, ...]` — one entry per slot that has at least one free bay. This is the same overlap logic as the model constraint, so a slot the method offers can never fail the constraint moments later under normal (non-concurrent) use.

Concurrency: Row Locking

> Two customers can request the same slot in the same instant. A Python-level `@api.constrains` check alone is not enough under PostgreSQL's default READ COMMITTED isolation — both transactions can pass the pre-check before either commits.

The booking controller's `_lock_and_pick_bay(company, start, end)` issues `SELECT id FROM carwash_bay WHERE id IN (...) FOR UPDATE` against the branch's bay rows *before* re-reading existing appointments and picking a free bay. A second concurrent request blocks on that lock until the first request's transaction commits, then re-reads and correctly sees the first booking — so it picks a different bay, or reports the slot full, instead of double-booking. This is plain PostgreSQL row locking; no new infrastructure.

Controller Flow (no JavaScript)

The whole booking flow is server-rendered, plain HTML forms — GET → GET → POST → GET — deliberately avoiding a JS/ajax layer, since the environment this was built in has no way to browser-test client-side behavior. Every route is `auth='user'`, so an anonymous visitor is auto-redirected to `/web/login` — this alone satisfies "portal login required to book."

Route	Method	Purpose
`/car-wash/book`	GET	Branch + service + date picker
`/car-wash/book/slots`	GET	Renders `_get_available_slots()` output + vehicle picker (existing or inline new-vehicle form)
`/car-wash/book/confirm`	POST (CSRF)	Locks bays, re-validates, `sudo()`-creates vehicle (if new) + appointment, calls the real `action_confirm()`
`/car-wash/book/success/<id>`	GET	Confirmation page

`/my/bookings`	GET	Portal list (CustomerPortal extension, mirrors `sale`'s `/my/orders` pattern)
`/my/bookings/<id>`	GET	Portal detail, via `_document_check_access`

Explicitly out of scope: online payment at booking time (billing still happens at completion via the existing Sales Order flow) and holiday/one-off closures (a clean upgrade path exists via `resource.calendar.leaves` if ever needed).

6. Integration Points

Odoo App	How It's Used
Sales (sale_management)	Every completed appointment and every membership billing cycle creates and confirms a real `sale.order` — all revenue reporting downstream is standard Odoo Sales/Accounting, nothing bespoke.
Stock	`action_done()` creates `stock.move` s for each service's consumables recipe (`carwash.service.material.line`), so material usage is real, reportable inventory movement.
Loyalty	Membership billing feeds the standard `loyalty.program` / `loyalty.card` models — points accrue on confirmed Sales Orders exactly as they would for any other customer.
Project + Timesheets	One shared `project.project` ("Car Wash Jobs"). Starting a job card creates a `project.task` ; passing QC logs one `account.analytic.line` timesheet entry — real, standard-Odoo-reportable time data (Timesheets app, project profitability), not just raw timestamp fields.
Website/Portal	Reuses `website.layout` and the `portal.CustomerPortal` extension pattern from `sale` — no bespoke front-end framework.
Resource (business hours)	`res.company.resource_calendar_id` powers slot generation — the same calendar model used for employee working schedules.

7. Installation & Operations

Install Order

Standard Odoo dependency resolution handles this automatically — installing `carwash_website` (the leaf module) pulls in all five others. For a minimal single-branch, no-online-booking deployment, install only `carwash_base` + `carwash_jobcard` (+ `carwash_membership` if recurring plans are needed).

Demo Data

Every module ships realistic demo data (customers, vehicles, bays, a full day's queue including one intentionally in-progress appointment, a second branch, an Owner and a Branch Manager account) so a fresh install is immediately explorable without manual setup.

Multi-Company Note for Developers

`env.companies` (active/switcher-selected companies) and `user.company_id` (the user's fixed home company) are **not the same thing** and are easy to conflate. Dashboard and booking scoping consistently use `env.companies` so behavior follows the company switcher, not just the user's default branch.

Regression Testing

There is no formal Odoo `tests/` package; verification throughout development used ad-hoc `odoo-bin shell` scripts exercising the full workflow per module plus targeted negative-path and cross-role access checks, re-run after every change against a clean reinstall.